BEE 4750 Homework 1: Introduction to Using Julia

Name: Evelyn Chen

ID: ec877

> **Due Date**
>
> Thursday, 9/11/25, 9:00pm

Overview

Instructions

- Problem 1 consist of a series of code snippets for you to interpret and debug. You will be asked to identify relevant error(s) and fix the code.
- Problem 2 asks you to write code to implement several simple functions using more general programming and some which are unique to Julia (which might mean that you need to look at the documentation or use Julia's help function, which is accessed with `?`, e.g. `?sum` for help with the `sum()` function) or to analyze Julia syntax.
- Problem 3 asks you to convert a verbal description of a wastewater treatment system into a Julia function, and then to use that function to explore the impact of different wastewater allocation strategies.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```
  Activating project at `~/BEE_4750/bee4750-hw1-ec8771`
```

Standard Julia practice is to load all needed packages at the top of a file. If you need to load any additional packages in any assignments beyond those which are loaded by default, feel free to add a `using` statement, though you may need to install the package.

```
using Random
using Plots
using GraphRecipes
using LaTeXStrings
using Distributions
```

```
# this sets a random seed, which ensures reproducibility of random
number generation. You should always set a seed when working with
random numbers.
Random.seed!(1)
```

```
TaskLocalRNG()
```

Problems (Total: 30 Points)

Problem 1 (12 points)

The following subproblems all involve code snippets that require debugging. You are encouraged to use online resources (*e.g.* Julia documentation and forums, Stack Overflow, Reddit, etc) to help find diagnose error messages and find solutions, but make sure that you clearly document which resources you used and how you used them.

Problem 1.1

You've been tasked with writing code to identify the minimum value in an array. You cannot use a predefined function. Your colleague suggested the function below, but it does not return the minimum value.

```
function minimum_new(array)
    # initialize the minimum value counter
    min_value = array[1] # min value is the first element in the array
    # update minimum values
    for i in 2:length(array) # modified the range to start from the
second element
        if array[i] < min_value
            min_value = array[i]
        end
    end
    # return found minimum
    return min_value
end

array_values = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
@show minimum_new(array_values);
```

```
minimum_new(array_values) = 78
```

What is the logical flaw in the code? Fix it and show that your fixed code solves the problem.

The error: With the original code, the function will not work correctly if all the elements in the array parameter are positive values (AKA greater than 0). Thus, I modified the min_value to take on the first element in the array. NOTE: Because minimum() is already a function in Julia and running minimum(array) was not working, I modified the function name to be minimum_new.

Problem 1.2

Your team is trying to compute the average grade for your class, but the following code produces an `UndefVarError`.

```
# enter student grade vector
student_grades = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
# compute class average

function class_average(grades)
  average_grade = mean(grades) # modified student_grades to grades
  return average_grade
end

@show class_average(student_grades);

class_average(student_grades) = 94.4
```

What does this `UndefVarError` mean? Why does this code produce one? Fix the error and solve the problem.

The error: Since the given parameter is grades, the function should refer to the parameter as grades, not student_grades. Thus, I modified the name in the function from student_grades to grades. In addition, I modified the @show average_grade to @show class_average(student_grades) to actually run the function and refer to the array student_grades in Main.

Problem 1.3

Your team wants to know the expected payout of an old Italian dice game called *passadieci* (which was analyzed by Galileo as one of the first examples of a rigorous study of probability). The goal of passadieci is to get at least an 11 from rolling three fair, six-sided dice. Your strategy is to compute the average wins from 1,000 trials, but the code you've written below produces a `MethodError`.

```
# function to simulate a passadieci roll: 3 6-sided dice

function passadieci()
    # this rand() call samples 3 values from the vector [1, 6]
    roll = rand(1:6, 3)
    return roll
end
# set number of trials and initialize outcome vector
n_trials = 1_000
outcomes = zeros(n_trials) # modified zero() to zeros() function
# simulate number of passadieci rolls and count wins
for i = 1:n_trials
    outcomes[i] = (sum(passadieci()) >= 11)
end
```

```
win_prob = sum(outcomes) / n_trials # compute average number of wins
@show win_prob;

win_prob = 0.493
```

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

The error: This MethodError is occurring because the arguments do not match the method's requried type parameters. The code is attempting to retrieve/set an index of a type that is not index-based (eg. outcomes, after using zero(), is a numerical value, not an array. I modified the zero() function to the zeros() function instead, ensuring that outcomes is an array of 1000 zeros, not just the value zero.

Problem 1.4

You're interested in writing some code to remove the mean of a vector from all of its components. You've written the following code and tried to test it on a random vector, but your code returns a `MethodError`.

```
# function to remove mean from a vector
function remove_mean(vect)
    # fucntion to compute the mean
    function compute_mean(vect)
        element_sum = 0 # initialize sum
        # compute mean and return
        for v in vect
            element_sum += v
        end
        return element_sum / length(vect)

    end

    m = compute_mean(vect) # compute mean
    # return demeaned vector

    # created for loop to individually remove mean from each component
in array
    for i in 1:length(vect) # iterate through the array
        vect[i] = vect[i] - m # remove m from each element
    end
    return vect
end

random_vect = rand(1_000)
print()
@show remove_mean(random_vect)

remove_mean(random_vect) = [-0.48714201347384456, -
0.12269216958726825, 0.4548715202104274, -0.028396677324517006,
```

0.34088759745318487, 0.08368502890520746, -0.3128216458197546, -0.3745114360039007, -0.21849086424244257, -0.40528185357328106, -0.28556783229938687, 0.468680463619218, -0.300790028683001, -0.20952041480451256, 0.35311012601272995, 0.1571804162488315, 0.29888873193645404, -0.11238194768236232, -0.3859918469522362, -0.08067362943634038, -0.4224255064298773, 0.22568452805139905, -0.07065480332076535, -0.04785543152138816, 0.2837802179198743, -0.27000743069327526, 0.04882550115228812, 0.16025664259571182, 0.32274973315765476, 0.46482176159768407, -0.2846132636965564, -0.4705365840931567, -0.19364144734834576, -0.08071506699006381, 0.38333514271463676, 0.42575592258134376, 0.17374899366812113, -0.07104077496796302, -0.25265748749249073, -0.124038341266687, -0.33866817296393337, -0.3237999204667025, -0.3432933944594889, 0.13695788178977097, 0.34771950023061504, 0.19369458059135714, 0.020492899688501653, 0.32497368125333737, -0.08480638243949801, 0.4747884842775705, -0.03380828389873736, 0.2944799713092263, 0.2940387751954755, -0.01837162520326685, 0.4987564085960776, 0.2619211137367019, 0.2233841899573804, 0.49293940588965823, 0.2919876137631756, 0.39302357628273954, -0.38934709100619447, 0.18954185170046034, 0.013079548732811475, 0.2760980500731356, -0.21800219281215605, 0.3976045490935062, -0.1066935992912067, 0.2559610185182163, -0.23366034731127927, 0.2781806183524125, -0.12547608726075787, -0.12010701477531316, 0.392357382398951, 0.14752428095733883, -0.39355069808633614, 0.26774524413154044, -0.3961072010171207, -0.060128914157950186, 0.28249645043466987, -0.11930109386375698, 0.04903969602537894, 0.010870008630925687, -0.33664050183475014, 0.30646001178184956, -0.4568353264538355, 0.012617205046733582, 0.0890004351518634, 0.12850691698766092, 0.4737456675566839, 0.1035929571275106, 0.4006804232343345, -0.2980265819604835, -0.02078086593304418, -0.39293026533407116, -0.21965195703022233, 0.04417540185579705, 0.4173984241965839, -0.35012029473580686, -0.06687299488743836, -0.05821254901774242, -0.2070806732202083, 0.1754012143629684, 0.011062458175650103, -0.44357452558600274, -0.30187855356344595, 0.28845803961489624, 0.4980809869238445, 0.013267697957391023, 0.4955560467631984, 0.07718013569514737, 0.3096187752305789, -0.01632720315167524, -0.14650843784860312, -0.35595232778147323, -0.14617247016537394, 0.3977467476960086, 0.08098334482582925, 0.02916859351548895, 0.13106004758230994, -0.23562482640510252, 0.12930918141663572, -0.11358729677271517, 0.4672492855621966, -0.388650360934343, 0.19468659313330428, -0.09096939669025339, 0.47033766374048525, -0.3626999656217109, 0.07663029178104297, -0.4588034033080606, -0.18489034012148564, 0.01711520538558775, 0.03087981582925403, 0.3944824410898561, -0.48092961751167673, -0.29581598278188537, 0.24523479774457135, 0.37516344541803404, -0.2958074690941739, 0.2285993620122252, -0.3083863196093046, -0.4761937423874234, 0.28005748476443315, 0.13901102074137495, 0.1305677189801563, -0.39112681876389666, 0.11004543016049517, 0.2019935188252049, 0.035169175242239725, -0.33351135961851963, -0.15549835507993037,

0.06608367400213333, 0.305554450880576, 0.047718918828769996,
0.20451900573507564, -0.3627972471908423, 0.39050120202677185, -
0.005502993838511605, -0.1951262119622209, -0.06138522366513188, -
0.4630158379594248, 0.17743667508641225, -0.32037982882124694, -
0.4179166696753681, -0.3620122283399553, 0.4639944840859761, -
0.08406419828326905, 0.139707763860892, 0.01935649030319353, -
0.013628454372491383, -0.20470273675306738, 0.16312066160120475,
0.3902768788283376, 0.09108775536620706, 0.06200901654012925,
0.15012569009638488, -0.47714713233305184, 0.49757199795042073,
0.06729710505847797, -0.33882821229454274, 0.21075415137739584, -
0.17881084294434024, -0.05554160175715073, 0.005947408994178449, -
0.2252116716634993, 0.22641166634247856, 0.4188582331627778,
0.20118009161010386, -0.18455424767977913, -0.14829762288437942, -
0.12158413143996483, 0.3693448830222298, -0.23377100369836445,
0.21928988168620245, 0.2855588669471214, -0.480392121479301, -
0.06777255604980492, -0.13811809150569443, 0.09620552594450704, -
0.45197945603698986, 0.0627201809112865, -0.4003321515432694,
0.4003297974227361, -0.4239595570398341, 0.30387057053261046,
0.228904974385302, 0.028826639782548624, 0.4375070521790907, -
0.19019787534786825, -0.12758838252409976, 0.2405472385154226,
0.13438133720542722, -0.40119767517740246, -0.3909631189761088, -
0.15479933265927792, 0.1966929806910097, 0.48827000855629743,
0.21734698807497999, -0.45728899060165273, -0.18610979465990818,
0.43342443873924297, -0.03659881500318607, -0.4677955496158328, -
0.018717957436347077, 0.2422109874332936, 0.23117518643368484, -
0.12465903068843909, -0.4644086267004045, 0.20604438699410033, -
0.39356026135431554, 0.5090714340925284, 0.48351423576453423, -
0.2726374214369347, 0.1468205416756111, 0.46429079794928263, -
0.3957455228412876, 0.3138231129152613, 0.06863537415496457,
0.10475678255416443, -0.4890052552052405, 0.42581498407537277,
0.07093063110829745, -0.4619549973015601, -0.2240313771267889,
0.28566875830973737, -0.3399742791214093, 0.032525927978509395, -
0.3754949509122367, 0.42566680375202337, -0.016423175573912485, -
0.36641017298567546, -0.24226896524330688, -0.36061239993692396,
0.08336309221399918, 0.05235245549707579, 0.15415850540795195, -
0.006759754097163673, -0.30639799006513846, 0.34589338185419427, -
0.20240801420686927, 0.27092236121869984, -0.3986017102421384,
0.5026525486920133, 0.5020325280702553, -0.15370032541132272,
0.14901054875770103, 0.3799545600742684, -0.044542321088475556, -
0.24178266595712528, -0.45960370387851357, 0.5097212307027443,
0.19317216877975107, -0.4685967235905647, 0.34327993584340977,
0.4079308547013316, 0.3393117421493357, 0.4850310073654355, -
0.15502960116471665, -0.02685497891885391, -0.14590217646354953,
0.24673979333919527, -0.3137708719298522, -0.11868520739029464, -
0.021797258695897215, 0.42342526812828707, 0.2585087138544737,
0.40332547640246263, -0.47536540054256227, -0.09879829901424275, -
0.06028730429259088, 0.23895411625658414, -0.4641982258839916, -
0.015276900838523844, 0.09097086041587976, 0.24997694181386798,
0.05889555830558302, -0.028719570141978568, 0.4075429655088625,

0.15428407653724419, 0.4824431948848874, 0.35445883763460106, -0.22488928836209898, 0.4008268796379961, -0.47556671905329273, 0.010657250691866627, 0.054365764520456605, 0.274998304515245, -0.12169650628374051, -0.16137431060355278, 0.021856576540301598, 0.43680112461690346, -0.39652022577784907, -0.1236268997185912, 0.1571185695358246, 0.4841683504824108, 0.048878830789217065, 0.10643520544374008, 0.5022494465564604, 0.4742879841791683, -0.43490019836969285, -0.20190250716076596, 0.03937667390101207, -0.20456585745446743, -0.20943342896282202, -0.4752011371812729, -0.11536098235257741, -0.1223131167961542, -0.18836517613090664, -0.05391497456386518, -0.23735719457907634, 0.2234402964112352, 0.054881291010063005, 0.39612701586494203, -0.46194339831023756, -0.18211226924646218, 0.26712299128628525, 0.4721630481812401, -0.45027338710909404, -0.40408399782829474, -0.43883789068225, 0.4143041069068749, 0.3273259619748401, 0.3807181089604388, -0.24719530647435672, -0.09844636292712783, 0.028486166794535195, -0.2992834781778747, 0.2268874849037943, -0.29925076309621435, -0.38079305238734173, -0.2642728080725988, -0.12364680902739766, -0.08804443593591904, -0.20979338934881986, 0.1875278822801657, 0.12248415493671339, -0.4571705394258164, 0.1565991832435807, 0.22185163922621298, -0.39988671257242203, 0.30746451570312205, 0.04549467154912823, 0.29818093123553546, -0.030875650895996753, -0.08793615050643433, 0.32334807744568284, -0.08984226895510306, 0.335820413732408, -0.08995157751932614, -0.39359164885997244, 0.2240550689815981, 0.23051258844792488, 0.4780871513051429, 0.21318675000726084, 0.03983870092314579, 0.30231048135854544, -0.31042828460687977, 0.006145110863616243, -0.03571679094426827, -0.2852288815972317, 0.2368356096321127, -0.43955103148351893, 0.11583505358419643, 0.2528308036761516, 0.3742578951227159, 0.441958960739829, 0.33761822651099727, -0.20522440783632634, -0.36078173539501524, -0.087394763521909, -0.27001283659768904, -0.41472936859677867, 0.46306112098436847, -0.2919342845678631, -0.1433015340144297, -0.12499259520714334, -0.15309566270319896, 0.30268974659075165, 0.20823469744298828, 0.3678293069022698, -0.2688697446468963, 0.0207006062275239, -0.2444095304873698, -0.2226119261940196, 0.39598189275507145, 0.3174386358983937, 0.40315406891879135, 0.09960808873880156, -0.23814151483675405, -0.39767357896096445, 0.08110890925627467, 0.1683350089812965, -0.4361764689640353, -0.21277383639814762, -0.3480418631286838, 0.3519263216030283, 0.16527465768861827, -0.4779058848321419, 0.09026877563982327, 0.36444020824411516, 0.36712535229481724, 0.44729524316600766, 0.056297077929821304, 0.14938010040204175, -0.020230641249638026, 0.2660611501940786, -0.2067740156787654, 0.3839897143082706 7, -0.126948624468599, 0.06564220614178662, -0.4575106233466064, 0.3362956271019263, 0.2611182006294581, -0.32133822187864425, -0.27661685870569497, -0.19154713439261595, -0.2211247308996186, 0.16483832503790907, -0.19970924489938124, -0.022276593282130275, 0.35206140353398097, 0.29732207193657123, -0.08498046003023119, 0.2446124194190541, -0.24847224829791137, -

0.4654990347388087, 0.4920400082745612, -0.2549844679768014, -0.0471213938134869, -0.48049291187627274, 0.18175747547132193, -0.03335009448032067, 0.06987663192077198, 0.46584633304736167, 0.11367809211203661, -0.08900959359547456, 0.1726575393667058, -0.04076321773994862, -0.08326797792316643, 0.12379230205729685, 0.16758342339167498, 0.370891660407053, 0.4525493972144673, -0.08852624233918427, -0.11785755701372802, -0.16326618547360694, -0.03815891180443165, 0.332178206736813, -0.16962323390901712, -0.4793932780261322, 0.23094944003744544, -0.40683137926093615, -0.4628989568361196, 0.30123873957375824, -0.2718683938865204, 0.34375341141681093, 0.07148907660766368, -0.17883133476471058, -0.10703752197465466, 0.2652417810055029, -0.009352407749228298, 0.12108266365533149, -0.10684641968696473, -0.30331391181119105, 0.3712278598840928, 0.03313404433361877, 0.49603101156796714, -0.16924134828108195, 0.27351654916059354, 0.1481730502814954, -0.42742102308198593, -0.2961419238490358, 0.1761245975395458, -0.1024454241213335, -0.26256107072857915, 0.3098118288620245, -0.0690384459622978, -0.09924957352912411, -0.2610981932172539, -0.4436561967940945, -0.1774316232759281, 0.06310135657562244, -0.20293757293019943, 0.02571440605992903, -0.34510534637043455, 0.08889634948294056, 0.4468108036595754, -0.2612885081378016, -0.15157233480847576, 0.29119893776881434, -0.44444049769566885, 0.22827823205604136, 0.13191440163504486, 0.051939779668781494, 0.5014244862955787, 0.3400630980545799, 0.0486574381178414, 0.1559083850221088, -0.2513952898765679, 0.26479368841990625, -0.34568581197177983, 0.1618168835480982, -0.03573004653373668, 0.10050250302891872, -0.4551838289681608, 0.0949375865557267, -0.4625430297295039, 0.20553710509478762, -0.48549760173657663, 0.2541872883054882, -0.10411213161077426, -0.12676576106046777, 0.1358644118148572, -0.40430112276423047, 0.4312672355356657, 0.3906509943363715, 0.3899721480169742, -0.22656512341197932, 0.12977723667350904, -0.12535747122117757, -0.4171184719727453, -0.3510920580836602, 0.48889138403217947, 0.3485725595073751, 0.08695744415274709, 0.4060930695444529, 0.3026634410084872, 0.482547775674655, 0.38396744606221433, 0.3828133376199857, -0.08352082016181284, 0.1505859243660186, -0.2983058093226062, -0.19829643705832456, -0.05563845245958199, -0.16605337277245058, -0.18091297013878738, 0.341011602599015, -0.3014329244240177, 0.11625090770293456, 0.5058895100570028, -0.4722934347162462, 0.43372966635117116, -0.3276593526146636, 0.2258411272423736, 0.42123850433358945, 0.0911351151965859, -0.4868696003549974, 0.3200937521511048, -0.07337144540092944, 0.27422881246168507, 0.09404961491330588, 0.08191856241525775, -0.030354622060317693, 0.43119032075321473, 0.14125438911373434, -0.2195580815484428, 0.3579844880061748, 0.04851914630568699, -0.12144597231419973, 0.1884743931861128, -0.48278338975597745, 0.49244640784250115, 0.34236759312815623, 0.1266620692013264, -0.11740914754621368, -0.11461604429864003, 0.06803872220505203, 0.20619420266356137, -0.14089701776352248, -0.14288512118762842, -0.1820364260023858,

0.09175503031850651, 0.03953379396624235, 0.16844639293233066, -0.3479598731815431, -0.08687079349632476, -0.46325901397185576, -0.19909031010945333, 0.3863910053952435, -0.12308036553493706, 0.10596773889111555, 0.3128964809962953, 0.07700426390020065, 0.18044982567373524, -0.3390196626528904, -0.45496281165100494, 0.2910406261224566, 0.2572942604240124, 0.4254519924922392, 0.3045447585891391, 0.3036042995447935, 0.2872347566444997, -0.28165270264160824, -0.36478815669401154, -0.44294712237070455, -0.11015045176519456, -0.18350843680352769, 0.03783831478380051, 0.45531510741528836, -0.44790820037809453, 0.30279357786126593, -0.28574756075566343, -0.323741626714501, 0.2176564284249205, 0.3166235819926464, -0.09874897584990772, -0.20233169582706179, -0.0241774493509459, -0.4092642056837059, -0.2833725552998607, 0.20077324724716084, -0.0816456372151394, 0.21651873949480016, 0.38234796143491934, 0.4961342745539812, 0.3107304936399341, -0.21305384334322908, -0.08245906750192855, -0.15820961889392582, -0.39845203593582157, 0.15478972391790313, 0.0692586017850243, -0.3657239058745059, -0.09030629266958412, -0.018694104419529567, -0.4860442143546383, -0.1826650276664894, 0.49911070588728834, -0.15016424930227334, -0.0678001472941121, 0.2718430415845623, -0.383780626297873, -0.006491177394482461, -0.027138553273839294, 0.33421463186640266, 0.40996383554202276, 0.10805450615784984, 0.39346629575192393, 0.14245924275274935, -0.3115321751999388, -0.4465948989201777, -0.23132155764320483, 0.12635890841188135, 0.29787838786157006, 0.22380578191155465, -0.19658013420798004, 0.37594669759892296, -0.23151154370922067, -0.2889902429495169, 0.00200595024861272, -0.04923906211115614, 0.47605148451149626, 0.3778652030499198, 0.2583306984206394, 0.3359617685438999, -0.028824222006317624, -0.010778667305543221, -0.21154373795116554, 0.4680421963245043, -0.2910955010153452, 0.17149589532882326, 0.05897612832518917, 0.4583763035206666, -0.15226034468909522, -0.43357727433941484, -0.34658189563725783, -0.30464638143734935, -0.0613914534364024, 0.41658323232188843, 0.15144359543006247, 0.061950385841065936, 0.12525362897275627, -0.0792443130028967, -0.23484561985855357, 0.3336798590132708, 0.16591469878625698, -0.37377740318172625, -0.2472583714126506, -0.2675356411530444, -0.3579666270762758, 0.3399155532745437, 0.45641550896817806, -0.03231175951290255, -0.1221789326628443, -0.4202113684390717, 0.38184564031345813, 0.423061599294569, 0.16849938855072832, 0.0718860321380782, -0.22554051363916594, 0.4166270365488557, -0.16503657732614796, -0.4830762775829587, -0.01893177365775378, -0.44652688216929604, 0.47396613347753713, 0.4176284084782589, 0.01459516094025426, -0.159313716230023, 0.11390401293016561, 0.3523998367737239, -0.12629499910085773, 0.34662746294680813, -0.1967998297473993, -0.1587781791525561, -0.20035074561023447, 0.49889760655550053, 0.45236704725553634, 0.027457337651071057, 0.33768582728845276, 0.1843394693413858, -0.10231153946837801, -0.4090226803097169, -0.1653420593697349, -0.16745579466632077, -0.41688492980067626, -0.414113309354992, 0.016019459185705243, -

0.27179905169487684, 0.1215425542554166, -0.20344037961537587,
0.2871761839177899, -0.3794906443100914, -0.4557227118215703, -
0.14036483830531477, 0.028108387286532532, 0.015009312147257425,
0.14576053441208048, 0.2875140012425923, 0.3807105073552711,
0.2806881780230158, 0.12532786717049138, -0.4755052432554695, -
0.1798753082129121, 0.11903835466321888, 0.06563934438708763, -
0.39462345244624664, -0.17344634951623994, -0.030351682030080984, -
0.291355042655253, 0.19866990681306396, 0.3905828970812355,
0.5002413650873128, -0.15362979064956317, -0.1700516955708682, -
0.04877325921031206, -0.12673744693403688, -0.372117306916673, -
0.20869173164106858, -0.48206065290307265, -0.3186855084643767,
0.37275416665138117, 0.33692641014167335, -0.19858104248785957,
0.3575362582712881, 0.11246063389280203, -0.1869488443488041, -
0.089072762288183 06, 0.25449875738946426, -0.3221800484760792, -
0.14922174000284405, 0.31136154055068666, 0.43137032450313406, -
0.3753099693186337, -0.43786576475556127, -0.05202492086045907, -
0.26636128029993833, 0.164878221208141, 0.011304563421305147, -
0.4585433982282081, 0.319908283260375, -0.07752671374571518,
0.13126769917671777, -0.23936849061766163, -0.390959305457665, -
0.48824609113046546, 0.38191120144037305, 0.10935292625306525,
0.14203392998212117, -0.09387175911995216, -0.07965660832911747, -
0.09839158365856082, -0.24618324946459802, -0.08988923786502534,
0.06604518194811804, 0.3969903025908078, 0.1622839183525574,
0.36786812546332603, 0.12064275540223135, -0.18208689348243945, -
0.3526272434233262, -0.2853618650932598, -0.2442598068382209,
0.16322144649522785, -0.07599288904498736, -0.14628527907400657,
0.24073037628364335, -0.10452129040540237, 0.37111422626297363, -
0.44257537336793806, 0.04491902131432435, -0.1327331233278718,
0.3095778574150162, -0.11080951207370549, -0.34659324888838006, -
0.43802495582541845, -0.008208160665916342, -0.17766082884479295, -
0.4479422919069148, -0.1508795973842575, 0.12390636652275255, -
0.09042285879336376, -0.07061218564978111, -0.20483208568308264,
0.5013435566990975, 0.09236748506886816, -0.3396388165780355, -
0.3708432355207524, 0.3322167339955332, 0.16523282712901421,
0.19000626552554345, 0.17254890154708358, 0.38871756991166295, -
0.1755043013855483, -0.2858699177686701, -0.3296123460829443, -
0.06555301223047016, -0.11389136022419044, 0.22298812255514022, -
0.20472545156874167, 0.18161031879221823, 0.08958409856744198, -
0.04737378694288863, -0.2133338827310205, -0.007013128828893045, -
0.3089469489606914, 0.4328028483236993, 0.383885800581937,
0.3566903459424454, -0.38078486327882377, 0.2997515361912847, -
0.48288252570424184, 0.04800770258413389, 0.014952143539107343, -
0.4379707463253574, 0.46804745011112525, -0.1734513655735891,
0.314831001766129, -0.24144911633253174, -0.1448325981122095, -
0.02020347222876162, -0.4511303710142056, 0.31548294067828375,
0.056364702757541796, -0.4590718626427477, 0.19482549156642526, -
0.06778968146453557, -0.10230704186552786, -0.315659818980051,
0.1674952770652025, -0.2424994091300533, -0.37443929794207476,
0.11688166096869912, 0.03290425988521861, 0.2642820829720868, -

```
0.4423773184777018, -0.0012986655272602121, -0.101621465791084,
0.35644162721623895, 0.1924775440711105, -0.4358764618811919,
0.23306416351063575, -0.05361762650036406, 0.1562299338104287, -
0.34265968122074086, -0.3589439147303649, -0.2261508413886949, -
0.3455057844544934, -0.3722169052344856, -0.19116613388458237,
0.048726723695244556, -0.09196611727983905, -0.33764019677357837, -
0.3980853757690014, -0.347259897588034, 0.4244049200313893, -
0.33863251871127265, 0.27138384596410037, 0.16901031583271098, -
0.08210850986661189, -0.24037987011078288, 0.18572416696131588, -
0.3660177880861336, 0.08344593857231286, -0.0004918528829029878, -
0.05292985306334275, 0.19623933867322274, 0.461114380306871, -
0.31244363185275703, 0.5054435941149689, -0.3352291686202207, -
0.32087387036697923, 0.15271032051247624, -0.26704111163534416,
0.024462780208012846, 0.2625355331228838, -0.4156572413496711,
0.12189688204604554, -0.06500076878779082, 0.0659013746193231, -
0.33434645460110035, 0.09175243270398703, 0.4781940145223248, -
0.2542695119650006, -0.19416599447703942, -0.46535501732313467,
0.5012484723129694, -0.08752486417742511, -0.0746326375449301,
0.35695637087944776, -0.1128840015847099, 0.33421486029048864, -
0.3028246005408435, 0.3729640122980733, 0.3155152769168932,
0.2785579066423066, -0.3134843192870429, 0.058663855306139934,
0.4937224744077253, 0.3275868204818261, 0.45817808137537186,
0.16049562208374957, -0.24321555055698474, -0.3431222196399488,
0.11269675391915968, -0.1591741372879545, 0.2049947946795878, -
0.38848750084246075, -0.20651614797136597, 0.48973827142460613,
0.03158485027702551, 0.29639875935019344, -0.31709528875730986, -
0.3709819215560366, 0.19623724237703932, 0.26143033512781366, -
0.14837210253035904, 0.014145932400742356, -0.18955074724251209, -
0.060327639364560315, -0.34058371016967937, -0.08252598471787964, -
0.4171505559505998, -0.4684398695982185, -0.29436926347278636, -
0.09102711781001149, -0.2935289868909273, 0.2677656872780496, -
0.2063665713470676, -0.2555868492492812, -0.40310022249261945, -
0.3630458542990115, 0.3540687809577501, -0.20518543437084857, -
0.3387400815818007, 0.2749883889359527, 0.07353081539718431, -
0.18892408861150445, -0.21335847028267252, 0.3136933279009799,
0.33289444587833794, -0.3110853871612481, -0.14108229789382365,
0.07444323536705044, -0.037947383322348704, -0.2600060456624331, -
0.35557351970923146, 0.07142798070451484, 0.005378492082023456]

1000-element Vector{Float64}:
 -0.48714201347384456
 -0.12269216958726825
  0.4548715202104274
 -0.028396677324517006
  0.34088759745318487
  0.08368502890520746
 -0.3128216458197546
 -0.3745114360039007
 -0.21849086424244257
 -0.40528185357328106
```

```
-0.28556783229938687
 0.468680463619218
-0.300790028683001
 ⋮
-0.18892408861150445
-0.21335847028267252
 0.3136933279009799
 0.33289444587833794
-0.3110853871612481
-0.14108229789382365
 0.07444323536705044
-0.037947383322348704
-0.2600060456624331
-0.35557351970923146
 0.07142798070451484
 0.005378492082023456
```

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

The error: This MethodError is occurring because an operation is used to connect two different types of arguments. In this case, the code is attempting to use an operation to subtract m (a numerical value) from vect (an array). I modified the code to instead iterate through the array (for loop) to subtract m from each individual element in the array.

Problem 2 (18 points)

Cheap Plastic Products, Inc. is operating a plant that produces $100\,\mathrm{m}^3/\mathrm{day}$ of wastewater that is discharged into Pristine Brook. The wastewater contains $1\,\mathrm{kg/m}^3$ of YUK, a toxic substance. The US Environmental Protection Agency has imposed an effluent standard on the plant prohibiting discharge of more than $20\,\mathrm{kg/day}$ of YUK into Pristine Brook.

Cheap Plastic Products has analyzed two methods for reducing its discharges of YUK. Method 1 is land disposal, which costs $X_1^2/20$ dollars per day, where X_1 is the amount of wastewater disposed of on the land ($\mathrm{m}^3/\mathrm{day}$). With this method, 20% of the YUK applied to the land will eventually drain into the stream (*i.e.*, 80% of the YUK is removed by the soil).

Method 2 is a chemical treatment procedure which costs \$1.50 per m^3 of wastewater treated. The chemical treatment has an efficiency of $e = 1 - 0.005\,X_2$, where X_2 is the quantity of wastewater ($\mathrm{m}^3/\mathrm{day}$) treated. For example, if $X_2 = 50\,\mathrm{m}^3/\mathrm{day}$, then $e = 1 - 0.005(50) = 0.75$, so that 75% of the YUK is removed.

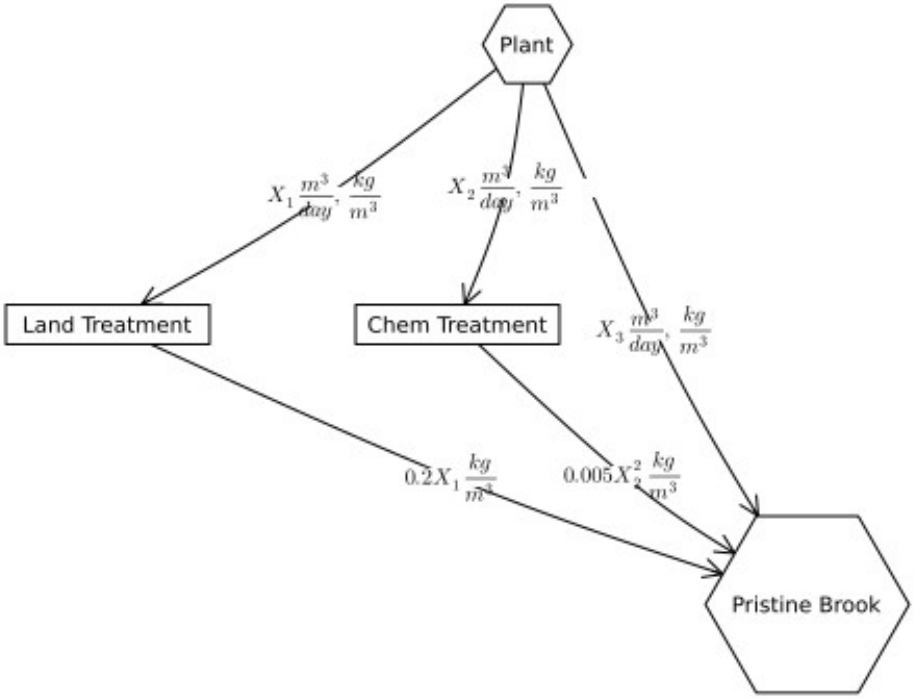
Cheap Plastic Products is wondering how to allocate their wastewater between these three disposal and treatment methods (land disposal, chemical treatment, and direct disposal) to meet the effluent standard while keeping costs manageable.

The flow of wastewater through this treatment system is shown in Figure 1. Modify the edge labels (by editing the `edge_labels` dictionary in the code producing Figure 1) to show how the wastewater allocations result in the final YUK discharge into Pristine Brook. For the

`edge_label` dictionary, the tuple (i, j) corresponds to the arrow going from node i to node j. The syntax for any entry is `(i, j) => "label text"`, and the label text can include mathematical notation if the string is prefaced with an `L`, as in `L"x_1"` will produce x_1.

Explanation: X_1 represents the amount of wastewater ($\frac{m^3}{day}$) and YUK ($\frac{kg}{m^3}$) sent for land treatment, X_2 represents the amount of wastewater ($\frac{m^3}{day}$) and YUK ($\frac{kg}{m^3}$) sent for chemical treatment, and X_3 represents the amount of wastewater ($\frac{m^3}{day}$) and YUK ($\frac{kg}{m^3}$) directly disposed in Pristine Brook. The YUK concentration after undergoing land treatment is $0.2\ X_1\ \frac{kg}{m^3}$, and the YUK concentration after undergoing chemical treatment is $0.005\ X_2^2\ \frac{kg}{m^3}$.

```
A = [0 1 1 1;
    0 0 0 1;
    0 0 0 1;
    0 0 0 0]

names = ["Plant", "Land Treatment", "Chem Treatment", "Pristine
Brook"]
# modify this dictionary to add labels
edge_labels = Dict((1, 2) => L"X_1 \frac{m^3}{day}, \frac{kg}{m^3}",
(1,3) => L"X_2 \frac{m^3}{day}, \frac{kg}{m^3}", (1, 4) => L"X_3 \
frac{m^3}{day}, \frac{kg}{m^3}",(2, 4) => L"0.2 X_1 \frac{kg}{m^3}",
(3, 4) => L"0.005 X_2^2 \frac{kg}{m^3}")
shapes=[:hexagon, :rect, :rect, :hexagon]
xpos = [0, -1.5, -0.25, 1]
ypos = [1, 0, 0, -1]

p = graphplot(A, names=names,edgelabel=edge_labels, markersize=0.15,
markershapes=shapes, markercolor=:white, x=xpos, y=ypos)
display(p)
```



Problem 2.1

Formulate a mathematical model for the treatment cost and the amount of YUK that will be discharged into Pristine Brook based on the wastewater allocations. This is best done with some equations and supporting text explaining the derivation. Make sure you include, as additional equations in the model, any needed constraints on relevant values. You can find some basics on writing mathematical equations using the LaTeX typesetting syntax here, and a cheatsheet with LaTeX commands can be found on the course website's Resources page.

Before formulating the equations for the treatment cost and amount of YUK discharged into Pristine Brook, the total amount of flow into Pristine Brook is $X_1 \mathrm{m}^3$/day + $X_{2} \text{m}^3\ text{/day} $ + $X_3 \mathrm{m}^3$/day = 100. X_1 is the amount of wastewater sent to land disposal, X_2 is the amount of wastewater sent to be chemically treated, and X_3 is the amount of wastewater sent to Pristine Brook. All of this totals to $100 \mathrm{m}^3$/day, the amount of wastewater that Cheap Plastic Products, Inc. is producing per day. Since the YUK concentration in the wastewater is $ 1 \ frac{kg}{m^3} $, our $\frac{m^3}{day}$ of wastewater and $\frac{kg}{m^3} $ of YUK is a one-to-one ratio.

Total Amount of YUK discharged into Pristine Brook:

We need to consider the amount of YUK discharged into Pristine Brook from all three outlets/methods: land treatment, chemical treatment, and direct disposal.

$Y_{landtreatment} = 0.2\,X_1$. With the land treatment method, 80% of YUK would be removed, so 20% of YUK would remain.

The formula for determining the amount of YUK removed with the chemical treatment method is e(X_2 ¿= 1 - $0.005\,X_2$. With the chemical treatment method, the percentage of YUK is dependent on the amount of YUK being treated, with the efficiency fator being $1 - 0.005\,X_2$. Thus, the equation for the amount of YUK remaining after the chemical treatment is $Y_{chemicaltreatment} = X_2\,(0.005\,X_2) \Rightarrow Y_{chemicaltreatment} = 0.005\,X_2^2$.

The third disposal method is directly discharging the YUK into Pristine Brook, which is defined by $Y_{direct} = X_3$.

This means the total amount of YUK that is discharged into Pristine Brook is $F(X_1, X_2, X_3) = 0.2\,X_1 + 0.005\,X_2^2 + X_3$.

$F(X_1, X_2, X_3) \leq 20\,\frac{kg}{day}$, according to the EPA's standard.

Total Treatment Cost $F(X_1, X_2, X_3)$:

We can look at the cost of each of the three treatment methods. For the first treatment method ($Y_{landtreatment}$), the daily cost is $\frac{X_1^2}{20}$. For the second treatment method ($Y_{chemicaltreatment}$), the daily cost is $1.5\,X_2$. For the third treatment option (Y_{direct}), there are no costs associated, since the power plant is simply discharging the wastewater directly into Pristine Brook.

Thus, the total daily cost function is $F(X_1, X_2, X_3) = \frac{X_1^2}{20} + 1.5\,X_2$.

Problem 2.2

Implement your systems model as a Julia function which computes the resulting YUK discharge and cost for a particular treatment plan. You can return multiple values from a function with a tuple, as in:

```
function multiple_return_values(x, y)
    return (x+y, x*y)
end

a, b = multiple_return_values(2, 5)
@show a;
@show b;


# my function starts here
function treatment_plan(X1, X2, X3)

    # part I: calculate the YUK discharge  (kg/day)
    discharge1 = 0.2*X1
```

```
    discharge2 = 0.005*X2^2
    discharge3 = X3
    discharge_tot = discharge1 + discharge2 + discharge3

    # part II: calculate the cost
    cost1 = X1^2/20
    cost2 = 1.5*X2
    cost3 = 0
    cost_tot = cost1 + cost2 + cost3

    return (discharge_tot, cost_tot)

end

# test case?
a, b = treatment_plan(55, 40, 5)
@show a;
@show b;
```

```
a = 7
b = 10
a = 24.0
b = 211.25
```

To evalute the function over vectors of inputs, you can *broadcast* the function by adding a decimal `.` before the function arguments and accessing the resulting values by writing a *comprehension* to loop over the individual outputs in the vector:

```
x = [1, 2, 3, 4, 5]
y = [6, 7, 8, 9, 10]

output = multiple_return_values.(x, y)
a = [out[1] for out in output]
b = [out[2] for out in output]
@show a;
@show b;
```

```
a = [7, 9, 11, 13, 15]
b = [6, 14, 24, 36, 50]
```

Make sure you comment your code appropriately to make it clear what is going on and why.

Problem 2.3

Use your function to experiment with 1,000 different combinations of wastewater discharge and treatment. You can do this with either a grid search or by sampling from a Dirichlet distribution (a $\text{Dirichlet}(1, n)$ distribution will generate uniformly-weighted n-dimensional vectors whose components add up to 1; see the `Distributions.jl` documentation for how to sample from probability distributions in Julia). Plot the results of these experiments. Do any satisfy the YUK

effluent standard (plot this as well as a dashed red line). What was the cost of solutions satisfying the standard? What can you say about the tradeoff between treatment cost and YUK discharge? You don't have to find an "optimal" solution to this problem, but what do you think would be needed to find a better solution?

See below for the code + plot. According to the graph generated, there are some combinations of wastewater discharge and treatment that do satify the YUK effluent standard as set by the EPA, though the majority of combinations do not. The cost of solutions satisfying the standard range approximately from $280 to $400. The tradeoff between treatment cost and YUK discharge follows an exponential decay trend. The greater the YUK discharge, the lower the cost, which makes logical sense since the land and chemical treatments have costs associated with these two methods, while direct disposal is free. In addtion, the greater the YUK discharge, the fewer treatment plan combinations there are (and the closer the cost ranges are to meet that YUk discharge). To find better solutions, we would have to find a combination that minimizes cost while satisfying the YUK discharge standard, likely through some mathematical constraints and models.

Problem 2.4

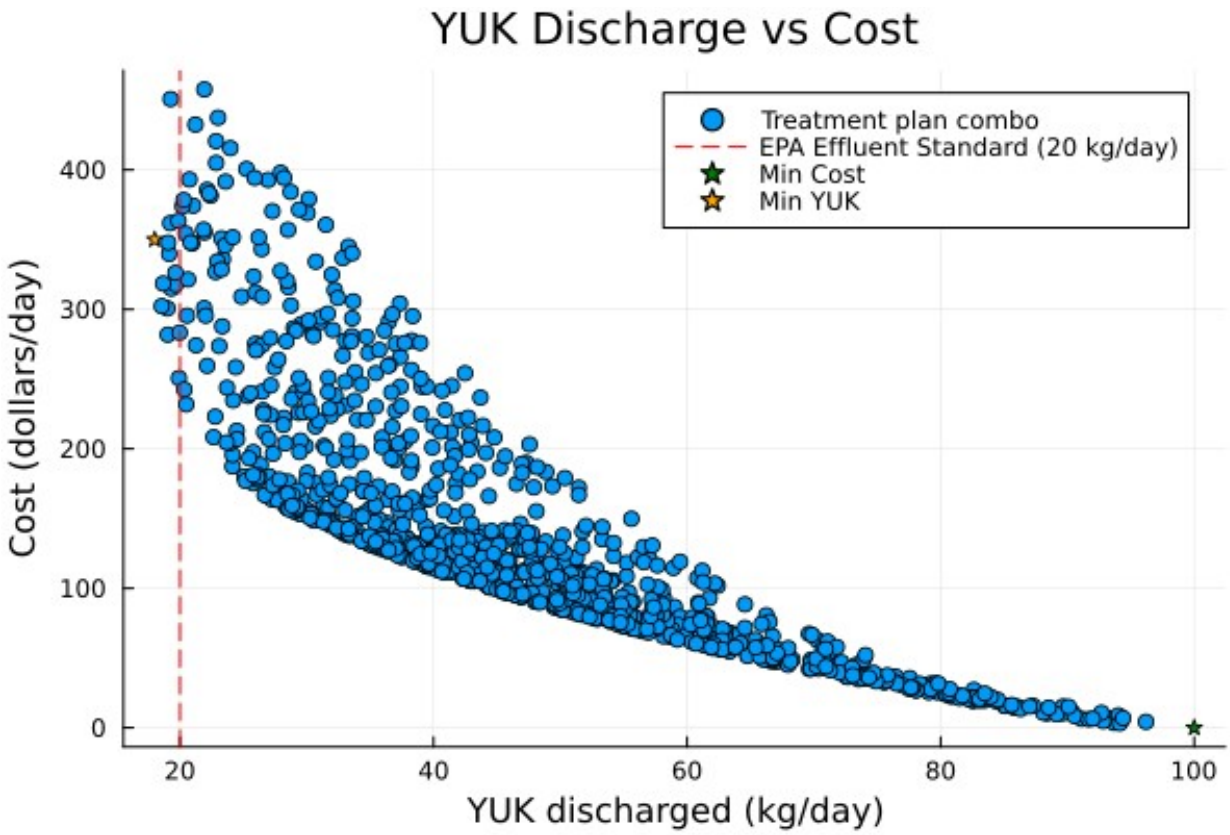
Find the strategies which minimize cost and YUK discharge (these will be different strategies) analytically and find the values of the objective metrics. Plot these values in the plot that you created for Problem 2.3. How do their values compare to the spread of values that you found in that problem? Would you select either of them (explain why or why not)?

The strategy to minimize cost would be to send all wastewater directly to Pristine Brook ($100\,\frac{m^3}{day}$), since this is the only free method. This would mean that none of the wastewater undergo the other treatment plans (land treatment and chemical treatment), which do cost money. The strategy to minimize YUk discharge would lkely be to find the combination of treatment plans that is most efficient at filtering out the YUK discharge, AKA a combination of the land treatment and chemical treatment. I began substituting in combinations for both land treatment and chemical treatment, and discovered that $(0.2)(80)+0.005(20)^2$ equals 18, which is the least amount of YUK discharged. Trying a combination of 81% land treatment + 19% chemical treatment, as well as a combination of 79% land treatment and 21% chemical treatment gives us a greater YUK discharge amount, so 80% landtreatment and 20% chemical treatment is the most efficient combination in minimizing YUK discharge. See plot to view the values of both strategies. These values are at the extreme ends of the spread of values in the graph. While it is more ethical to select the option that minimizes YUK discharge over minimizing cost, the most ideal option would be to find the least expensive combination that would still satisfy the EPA effluent standard (estimated to be roughly $280, according to the graph).

References

List any external references consulted, including classmates. Other than the references mentioned in this lab, I also consulted documentation on Julia plots (specifically the scatter function and its the vline function, as well as legend markers and the push function) for 2.3 and 2.4. I did this by searching up "scatter function," "vline function", "markers color and legend julia" to learn more about their implementation.

```julia
Random.seed!(123) # setting random seed

# sample 1000 different combos from Dirichlet(1,3)
combos = rand(Dirichlet(3, 1.0), 1000) .* 100 # scale it to 100
# note: samples 1000 vectors, 3-D Dirichlet disribution with
concentration parameters = 1
# sum to 1 -> sum to 100


arr_YUK = Float64[]   # store YUK discharges in an array
arr_costs = Float64[]   # store costs in an array

for i in 1:1000
    # pull out the combos by viewing each column while reiterating
through the 1000 samples
    X1 = combos[1, i]
    X2 = combos[2, i]
    X3 = combos[3, i]

    YUK, costs = treatment_plan(X1, X2, X3) # run the model

    # add to arrays
    push!(arr_YUK, YUK)
    push!(arr_costs, costs)
end

# plot results
scatter(arr_YUK, arr_costs,
    xlabel="YUK discharged (kg/day)",
    ylabel="Cost (dollars/day)",
    title="YUK Discharge vs Cost",
    label="Treatment plan combo")

# add the EPA's effluent standard line (20 kg/day)
vline!([20], color=:red, linestyle=:dash, label="EPA Effluent Standard
(20 kg/day)")

# min-cost strategy
minYUK, mincost = treatment_plan(0, 0, 100)
scatter!([minYUK], [mincost], color=:green, marker=:star, label="Min
Cost")

# min-YUK strategy
minYUK, mincost = treatment_plan(80, 20, 0)
scatter!([minYUK], [mincost], color=:orange, marker=:star, label="Min
YUK")
```


YUK Discharge vs Cost
Treatment plan combo
EPA Effluent Standard (20 kg/day)
Min Cost
Min YUK
Cost (dollars/day)
0
100
200
300
400
20
40
60
80
100
YUK discharged (kg/day)